

Análisis de Lenguajes de Programación

Programando con Lambda cálculo

23 de Septiembre de 2026

Programando en λ -cálculo

- ▶ Vimos que con λ -cálculo (LC) podemos representar cualquier función computable.
- ▶ La primer pregunta que surge es, ¿cómo podemos usar λ -cálculo para escribir programas?
- ▶ Codificaremos los valores (booleanos, naturales, listas, etc) mediante formas normales.
- ▶ Veremos dos formas de escribir funciones recursivas.

Booleanos

- ▶ Necesitamos codificar los valores *True* y *False* (constructores) y el operador *ifthenelse* (eliminador).
- ▶ Esta codificación debe satisfacer la siguiente especificación:

$$\begin{aligned} \text{ifthenelse } \text{True } P \ Q &=_{\beta} P \\ \text{ifthenelse } \text{False } P \ Q &=_{\beta} Q \end{aligned}$$

- ▶ Es decir:

$$\begin{aligned} \text{ifthenelse } \text{True} &=_{\beta} \lambda p \ q . p \\ \text{ifthenelse } \text{False} &=_{\beta} \lambda p \ q . q \end{aligned}$$

- ▶ Una solución es la siguiente:

$$\begin{aligned} \text{ifthenelse} &\equiv \lambda x . x \\ \text{True} &\equiv \lambda p \ q . p \\ \text{False} &\equiv \lambda p \ q . q \end{aligned}$$

Funciones sobre booleanos

$$\begin{aligned} \text{Not} &\equiv \lambda b. \text{ifthenelse } b \text{ False True} \\ &\equiv \lambda b. (\lambda x. x) b (\lambda p q. q) (\lambda p q. p) \\ &\rightarrow_{\beta} \lambda b. b (\lambda p q. q) (\lambda p q. p) \end{aligned}$$

$$\begin{aligned} \text{And} &\equiv \lambda b c. \text{ifthenelse } b c \text{ False} \\ &\equiv \lambda b c. (\lambda x. x) b c (\lambda p q. q) \\ &\rightarrow_{\beta} \lambda b c. b c (\lambda p q. q) \end{aligned}$$

Ejercicio: Definir *Or*.

- ▶ Necesitamos codificar el constructor *Pair* y los eliminadores *fst* y *snd*.
- ▶ Esta codificación debe satisfacer la siguiente especificación:

$$\begin{aligned}fst (Pair\ X\ Y) &=_{\beta} X \\snd (Pair\ X\ Y) &=_{\beta} Y\end{aligned}$$

- ▶ Una solución es la siguiente:

$$\begin{aligned}Pair &\equiv \lambda x\ y. \lambda b. ifthenelse\ b\ x\ y \\fst &\equiv \lambda p. p\ True \\snd &\equiv \lambda p. p\ False\end{aligned}$$

Ejercicio: Verificar que las definiciones satisfacen la especificación.

Numerales de Church

Para codificar los naturales Church propuso la siguiente codificación:

$$\begin{aligned}\underline{0} &\equiv \lambda f x . x \\ \underline{1} &\equiv \lambda f x . f x \\ \underline{2} &\equiv \lambda f x . f (f x) \\ &\dots\end{aligned}$$

Definición: Para cada $n \in \mathbb{N}$, el numeral de Church para n , es un término $\underline{n}$ definido como:

$$\underline{n} \equiv \lambda f x . f^n x$$

donde f^n denota n aplicaciones de f .

Numerales de Church

Podemos definir la abreviación f^n en el metalenguaje como:

$$\begin{aligned} F^0 X &\equiv X \\ F^{n+1} X &\equiv F(F^n X) \end{aligned}$$

Notar que:

$$\underline{n} F X =_{\beta} F^n X$$

Esta β equivalencia nos ayudará a pensar las funciones sobre naturales:

$$Succ \equiv \lambda n . \lambda f x . f (n f x)$$

Notar que $\underline{0} \equiv False$

Funciones sobre naturales

$$iszero \equiv \lambda n . n (\lambda x . True) False$$

Veamos que es correcta:

$$\begin{aligned} iszero \underline{0} &\equiv (\lambda n . n (\lambda x . True) False) \underline{0} \\ &\rightarrow_{\beta} \underline{0} (\lambda x . True) False \\ &\equiv (\lambda f x . x) (\lambda x . True) False \\ &=_{\beta} False \end{aligned}$$

$$\begin{aligned} iszero \underline{n + 1} &\equiv (\lambda n . n (\lambda x . True) False) \underline{n + 1} \\ &\rightarrow_{\beta} \underline{n + 1} (\lambda x . True) False \\ &\equiv (\lambda f x . f^{n+1} x) (\lambda x . True) False \\ &\equiv (\lambda f x . f(f^n x)) (\lambda x . True) False \\ &=_{\beta} (\lambda f x . f(n f x)) (\lambda x . True) False \\ &=_{\beta} (\lambda x . True)(n (\lambda x . True) False) \\ &\rightarrow_{\beta} True \end{aligned}$$

Más funciones sobre naturales

$$Suma \equiv \lambda n m . n \text{ Succ } m$$

Ejercicio: Definir la función *Prod*, que calcula el producto de naturales.

Numerales de Church y foldn

- Dada la siguiente definición de *foldn* en Haskell:

$$\begin{aligned} \text{foldn} &:: \text{Nat} \rightarrow (a \rightarrow a) \rightarrow a \rightarrow a \\ \text{foldn } \text{Zero } s \ z &= z \\ \text{foldn } (\text{Suc } n) \ s \ z &= s \ (\text{foldn } n \ s \ z) \end{aligned}$$

- Si usamos esta función como eliminador de los naturales, obtenemos la siguiente especificación:

$$\begin{aligned} \text{foldn } \text{Zero} &=_{\beta} \lambda s \ z . z \\ \text{foldn } (\text{Suc } n) &=_{\beta} \lambda s \ z . s \ (\text{foldn } n \ s \ z) \end{aligned}$$

- Tomando $\text{foldn} = \lambda x . x$ y las definiciones de $\underline{0}$ y *Succ* dadas anteriormente, se obtiene una solución para la especificación dada.

Utilizaremos esta técnica para encontrar codificaciones que representen a los tipos de datos recursivos:

1. Identificar **constructores**.
2. Escribir el **fold** correspondiente al tipo de datos.
3. Usar la definición de **fold** como especificación.
4. Definir el **fold** como la función identidad y derivar a partir de las ecuaciones los λ -términos correspondientes a los constructores.

- ▶ Los constructores de listas son **nil** y **cons**.
- ▶ Escribimos el **fold** para listas en Haskell (reordenando los argumentos respecto al foldr del Preludio):

$$\begin{aligned}\text{foldr} &:: [a] \rightarrow (a \rightarrow b \rightarrow b) \rightarrow b \rightarrow b \\ \text{foldr } [] &c n = n \\ \text{foldr } (x : xs) &c n = c (\text{foldr } xs c n)\end{aligned}$$

- ▶ Usamos esta definición como especificación.

$$\begin{aligned}\text{foldr } Nil &=_{\beta} \lambda c n . n \\ \text{foldr } (Cons\ x\ xs) &=_{\beta} \lambda c n . c (\text{foldr } xs c n)\end{aligned}$$

- ▶ Obtenemos la siguiente solución:

$$\begin{aligned}\text{foldr} &\equiv \lambda x . x \\ Nil &\equiv \lambda c n . n \\ Cons &\equiv \lambda x\ xs . \lambda c n . c (xs\ c\ n)\end{aligned}$$

Funciones sobre listas

- ▶ Para escribir funciones sobre listas en λ -cálculo, conviene escribir primero su definición en Haskell en términos de *foldr*.
- ▶ Por ejemplo, escribimos la función *length*:

$$\text{length } xs = \text{foldr } xs (\backslash x \ r \rightarrow r + 1) \ 0$$

- ▶ Luego la reescribimos en λ -cálculo:

$$\text{length} \equiv \lambda xs . xs (\lambda x \ r . \text{Succ } r) \ \underline{0}$$

Ejercicio: Definir la función *map* en LC.

- ▶ Algunas funciones recursivas son difíciles de representar con *fold*.
- ▶ Por ejemplo, la función *tail* en Haskell se define con *foldr* usando la técnica de *tupling*:

$$\begin{aligned} aux &:: [a] \rightarrow ([a], [a]) \\ aux\ xs &= foldr\ xs\ (\backslash x\ (xs, ys) \rightarrow (x : xs, xs))\ ([], []) \end{aligned}$$
$$tail\ xs = snd\ (aux\ xs)$$

- ▶ En λ -cálculo:

$$\begin{aligned} aux &\equiv \lambda xs. xs\ (\lambda x\ p. Pair\ (Cons\ x\ (fst\ p))(snd\ p))\ (Pair\ Nil\ Nil) \\ tail &\equiv \lambda xs. snd\ (aux\ xs) \end{aligned}$$

Operador de punto fijo

Definición: Un operador de punto fijo, es un término F tal que

$$F B =_{\beta} B (F B)$$

Teorema El combinador Y definido como:

$$Y \equiv \lambda x. (\lambda y. x (y y)) (\lambda y. x (y y))$$

es un operador de punto fijo. Es decir, para todo término X :

$$Y X =_{\beta} X (Y X)$$

Decimos también que $Y X$ es un punto fijo de X .

Ejercicio: Probar el teorema.

Funciones recursivas utilizando Y

- ¿Por qué la siguiente definición de factorial no es válida en λ -cálculo?

$$fac \equiv \lambda n . ifthenelse (isZero\ n) \underline{1} (prod\ n\ (fact\ (pred\ n)))$$

- Si abstraemos la llamada recursiva obtenemos el siguiente término:

$$B \equiv \lambda f\ n . ifthenelse (isZero\ n) \underline{1} (prod\ n\ (f\ (pred\ n)))$$

- Con lo cual para encontrar una definición de *fact* deberíamos encontrar una solución a la ecuación:

$$fact =_{\beta} B\ fact$$

Funciones recursivas utilizando Y

Definimos $fact \equiv Y B$ y veremos que satisface la ecuación anterior:

$$fact \equiv Y B =_{\beta} B (Y B) \equiv B fact$$

Es decir:

$$fact \equiv Y (\lambda f n. (isZero n) \underline{1} (prod n (f (pred n))))$$

- ▶ Vimos representaciones en LC para booleanos y pares.
- ▶ Vimos representaciones de dos tipos recursivos: naturales y listas.
- ▶ Para los tipos recursivos usamos una representación en términos de la función *fold* asociada a cada tipo de datos.
- ▶ Las representaciones para cada uno de los tipos de datos no son únicas.
- ▶ Vimos que con combinadores de punto fijo podíamos representar en LC funciones recursivas.
- ▶ Concluimos que el LC es un lenguaje de programación

1. Lambda-Calculus and Combinators. J. R. Hindley and J. P. Seldin. Cambridge University Press (2008).
2. Theories of Programming Languages. J. Reynolds (1998).
3. Types and Programming Languages. B.C. Pierce (2002).